第 1 章　无线局域网隐藏和暴露终端仿真

1.1 实验目标

1)了解无线局域网隐藏和暴露终端的原理

2)掌握无线局域网中 RTS/CTS 工作过程

3)掌握 NS-2 中隐藏和暴露终端仿真分析

1.2 预备知识

1.2.1CSMA/CA 机制

在无线网络中，数据传输所消耗的能量较大，若发生冲突，则接收信号能量会增加 5-10%左右。为降低冲突概率，无线局域网采用了载波侦听多路访问冲突避免(Carrier Sense Multiple Access with Collision Avoid, CSMA/CA)机制，平衡了共享信道的有效使用与避免冲突的需求，在无线网络中实现可靠和公平的通信。CSMA/CA 关键特性如下：

①**载波侦听**：设备在发送前监听信道，以确保它当前没有被另一个设备使用；

②**多路访问**：多个设备共享同一信道，并且可以同时传输；

③**冲突避免**：如果有两个或更多设备试图同时发送，则会发生冲突，CSMA/CA 使用随机回退时间间隔来避免冲突；

④**确认(ACK)**：成功传输后，接收设备发送 ACK 以确认收到；

⑤**公平性**：确保所有设备都有平等的机会访问信道，信道不被独占；

⑥**二进制指数回退**：如果发生冲突，设备在尝试重新传输之前等待一个随机的时间段，每次重传尝试的回退时间指数增加；

⑦**帧间间隔(IFS)**：要求传输之间有最小的时间量，以允许信道清除，减少冲突的可能性；

⑧**RTS/CTS 握手**：使用请求发送(Request To Send, RTS)和清除发送(Clear To Send, CTS)握手，在传输前保留信道，减少冲突机会以提高效率。

作为 IEEE 802.11 无线局域网标准的基本媒介访问控制技术，分布式协调功能(Distributed Coordination Function, DCF)采用 CSMA/CA 和二进制指数回退算法。DCF 要求发送节点在 DCF 帧间间隔(DCF Interframe Space, DIFS)内监听信道状态。若在 DIFS 内发现信道忙，节点将推迟发送。在多个发送节点争用无线信道时，若有多个节点感知到信道忙，并推迟其访问，它们将几乎同时发现信道被释放，然后尝试抢占信道，结果可能会发生冲突。为避免此类冲突，DCF

仅用于无线网络技术实验

还指定了随机回退(如下图的随机争用阶段)，强制站点额外推迟其对信道的访问。常见的 3 种帧间间隔(Interframe Space, IFS)包括：优先级最高的短 IFS(Short IFS, SIFS)、优先级中等的点协调功能 IFS(Point Coordination Function IFS, PIFS)和优先级最低的 DIFS，SIFS 用于 ACK 和 RTS/CTS 的传输，DIFS 用于普通数据的传输，具体不同标准的 SIFS 时长如表 1 所示。DIFS 与 SIFS 的关系为：SIFS+ (2*Slot Time)；PIFS 与 SIFS 的关系为：SIFS+ Slot Time。

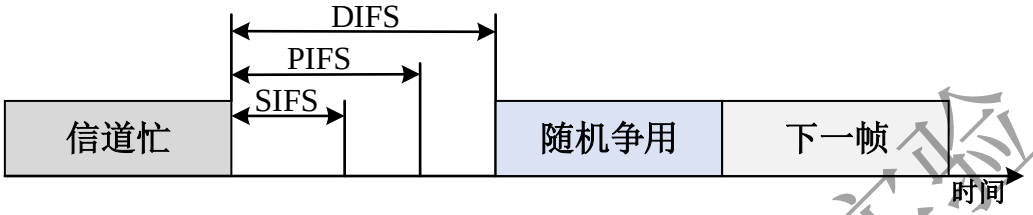


图 1.1　帧间间隔示意图

表 1.1　不同 IEEE 标准的 SIFS 时长

标准	SIFS(μs)
IEEE 802.11b	10
IEEE 802.11a	16
IEEE 802.11g	10
IEEE 802.11n (2.4 GHz)	10
IEEE 802.11n, IEEE 802.11ac (5 GHz), IEEE 802.11ax	16
IEEE 802.11ah (900 MHz)	160
IEEE 802.11ad (60 GHz)	3

除 DCF 机制外，IEEE 802.11 标准还定义了点协调功能(Point Coordination Function, PCF)的可选方法。PCF 允许作为网络协调器的接入点管理信道访问。IEEE 802.11e 标准修正案通过一个新的协调功能，称为混合协调功能(Hybrid Coordination Function, HCF)，增强了 DCF 和 PCF。

1.2.2 隐藏终端问题

无线局域网隐藏终端指处于接收端的覆盖范围内而在发送端的覆盖范围外的节点，由于监听不到发送端的数据传输，隐藏终端不受限制地发送分组到该接收节点，导致分组在接收端处冲突。如图 1.2 所示，**Node 1** 和 **Node 2** 互为隐藏节点，彼此不在通信范围内，且都向 AP 发送数据，致使 AP 收到的信息产生错误。

为解决隐藏终端问题，可采用 RTS/CTS 机制来避免冲突。发送方在数据发送前先发送一个 RTS 包，告知传输范围内的所有终端不要有任何发送操作。若接收方目前空闲，则响应一个 CTS 包，告诉发送方可开始发送数据，此 CTS 包也会告知在接收方信号传输范围内的所有其它节点不要进行任何传输操作。在 RTS 和 CTS 中都包含网络分配矢量(Network Allocation

仅用于无线网络技术实验

Vector，NAV)，用于表示后续数据传输需占用多少时间。图 1.3 所示为 RTS/CTS 和一次数据传输的过程，发送新的数据之前需等待 DIFS 时间。

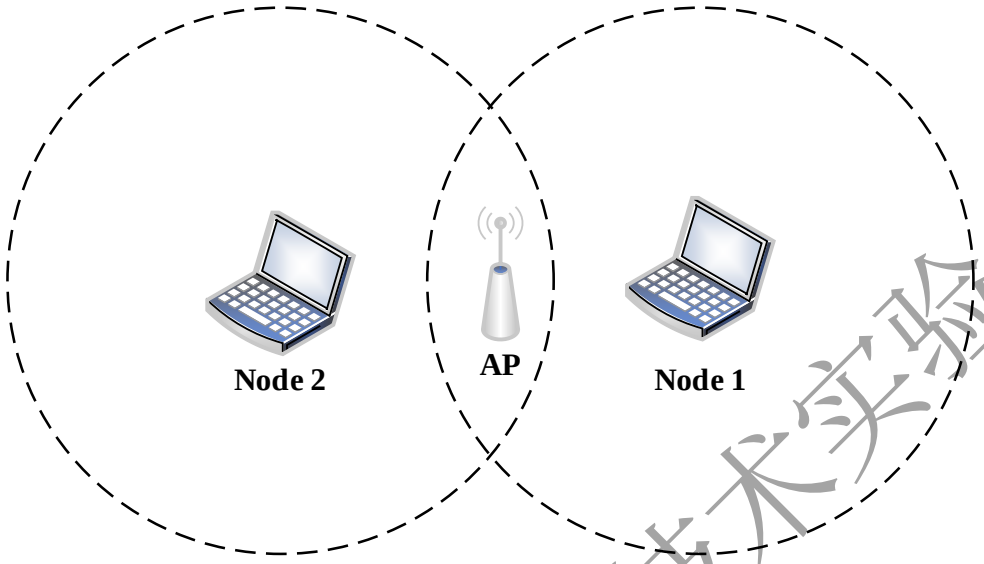


图 1.2　隐藏终端问题示意图

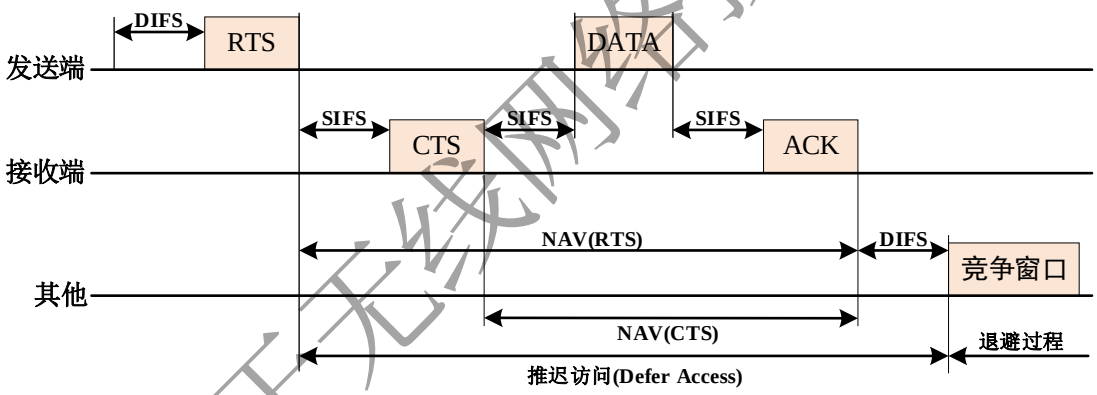


图 1.3　RTS/CTS 过程示意图

在整个 RTS/CTS 交互过程中会用到好几个帧，实际开始传输数据之前的延迟也会消耗一定的频宽。因此，RTS/CTS 常用于传输竞争比较显著的场合。在具体使用过程中，可通过调整 RTS 门限值(RTS Threshold)控制 RTS/CTS 功能，只有大于门限值才会执行 RTS/CTS 交换过程。早期 802.11 帧最大长度为 2346，RTS Threshold 的范围是 0-2347B。当设为 0 表示只要发送数据包，AP 就要先发送 RTS；当设为其他值，比如 400，此时大小为 410 的数据包发送时，路由就会发送 RTS 信号通知对方，以防信号冲突；当设为 2347，表示所有数据包直接发送，无需先发 RTS。

1.2.3 暴露终端问题

无线局域网暴露终端指在发送端覆盖范围内而在接收端覆盖范围外的节点，暴露终端因监听到发送端的发送而可能延迟发送。但它其实是在接收端的通信范围之外，它的发送不会造成

仅用于无线网络技术实验

冲突，这种延迟发送并不必要。

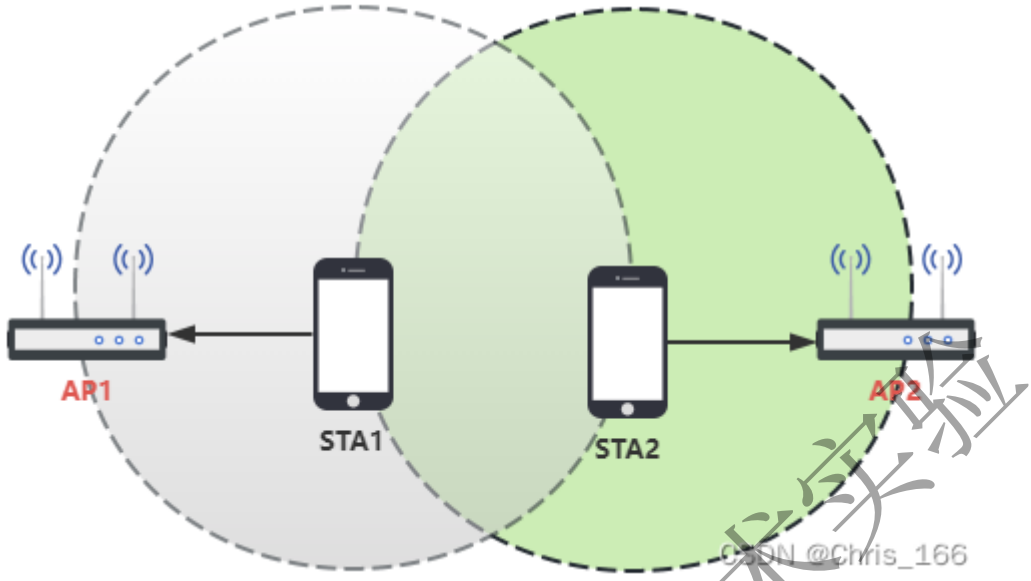


图 1.4　暴露终端问题示意图

物理载波监听引起的“暴露节点”问题：如上图，STA1 和 STA2 互相覆盖，即两者可以互相监听到对方的传输。但 AP1 仅仅能接收到 STA1 的信号，其不会受其他传输的干扰；AP2 也仅仅能收到 STA2 的信号，不会受到其他传输的干扰扰。在 LBT(Listen Before Transmission)机制中，若 STA1 向 AP1 传输数据，STA2 因监听到信道忙就不能向 AP2 进行传输。反之亦然。

虚拟载波引起的“暴露节点”问题：对于虚拟载波监听，即 NAV 机制，实际上也会引起暴露终端问题。在 Wi-Fi 中，节点和 AP 可以设置发送帧中的 Duration 字段，来对 NAV 进行设置。当 NAV 大于 0 的时候，节点或者 AP 的虚拟载波监听都会显示忙的状态，进而让节点认为信道就一直是忙，无法进行传输。只有当 NAV 等于 0 的时候，虚拟载波监听的结果才是空闲，此时才可以根据物理载波监听的结果判断信道是忙还是闲。很多时候，会看到 NAV 大于 0 的设置，此时也可被认为是一种暴露终端。

1.3 实验环境

该实验为无线局域网隐藏和暴露终端仿真，实验系统为 Ubuntu(版本为 20.04)，搭载在 VirtualBox(版本为 6.1)虚拟机之上，实验所用仿真软件为官方 NS-2，不需要进行功能扩展。对于实验环境安装有任何问题，请先完成本书的引导实验，其中包含了虚拟化软件、Ubuntu 系统和 NS-2 仿真软件的安装。此外，对于 NS-2 使用有任何问题可参考本书配套理论教材或 NS-2 官方文档，在此不再详述。

仅用于无线网络技术实验

1.4 实验步骤

本实验含两个子任务：①隐藏终端问题及 RTS/CTS；②暴露终端问题。需要注意，实验系统的用户名和主机名均为“wiotlab”，实验仿真工具和脚本在用户目录下，仿真工具完整路径为“**/home/wiotlab/WIoTLab/tools**”，实验脚本路径为“**/home/wiotlab/WIoTLab/exps**”。对于本实验，为脚本路径下的“lab1”文件夹。

为更好的完成本实验，需对 NS-2 无线物理层的几个关键参数进行解释，具体如下：

①载波侦听阈值(CSThresh_)：当分组的接收功率 Pr<CSThresh_时，接收不到该分组；而当 Pr>CSThresh_时，可以接收到该分组；

②接收阈值(RXThresh_)：当 CSThresh_<Pr<RXThresh_时，可接受到该分组，但因 Pr<RXThresh_而无法正确解码。若 Pr>RXThresh_，则该分组可以被正确解码；

③捕获效应阈值(CPThresh_)：当接收到碰撞的分组时，若较强信号是较弱信号的 CPThresh_倍，则较强信号仍可被正确解码。否则，接收到的分组都被丢弃。

通常载波侦听的范围需设置为信号传输范围的 2.2 倍，且 CSThresh_和 RXThresh_都是功率值(单位：瓦)。具体如何将距离换算成 CSThresh_和 RXThresh_值，可使用 NS-2 目录(“ns-2.35/indep-utils/propagation”)下的 threshold 工具(源代码请采用 g++编译)。以 NS-2 的“Propagation/TwoRayGround”传播模型为例，计算 200 米距离的值，如下：

```
$ ./threshold -m TwoRayGround -Pt 0.281838 200
distance = 200
propagation model: TwoRayGround

Selected parameters:
transmit power: 0.281838
frequency: 9.14e+08
transmit antenna gain: 1
receive antenna gain: 1
system loss: 1
transmit antenna height: 1.5
receive antenna height: 1.5

Receiving threshold RXThresh_ is: 8.91753e-10
```

上述输出中的最后一行的数值即为 200 米对应的 RXThresh_或 CSThresh_阈值，实验中请根据所设计的拓扑中节点间距离进行计算。若 RXThresh_和 CSThresh_设置为相同值，则表示关闭载波侦听功能。

仅用于无线网络技术实验

1.4.1 隐藏终端问题及 RTS/CTS

在隐藏终端问题仿真实验中，构建了如图 1.5 所示的拓扑，包含 3 个节点且相互间隔 50 米。其中，节点 0 向节点 1 发送数据(CBR/UDP)、节点 2 向节点 1 发送数据(CBR/UDP)。利用前述讨论的 threshold 工具计算得到 50 米的 CSThresh_和 RXThresh_值为 7.69113e-08，

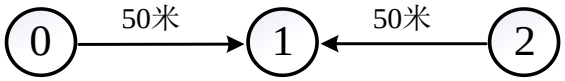


图 1.5　实验拓扑结构示意图

隐藏终端问题仿真分析的实验脚本如下：

```
#无线模块配置
Mac/802_11 set RTSThreshold_ 3000
Mac/802_11 set dataRate_ 2Mb
Antenna/OmniAntenna set X_ 0
Antenna/OmniAntenna set Y_ 0
Antenna/OmniAntenna set Z_ 1.5
Antenna/OmniAntenna set Gt_ 1.0
Antenna/OmniAntenna set Gr_ 1.0
Phy/WirelessPhy set CPThresh_ 10.0
Phy/WirelessPhy set CSThresh_ 7.69113e-08
Phy/WirelessPhy set RXThresh_ 7.69113e-08
Phy/WirelessPhy set bandwidth_ 2e6
Phy/WirelessPhy set Pt_ 0.281838
Phy/WirelessPhy set freq_ 9.14e+6
Phy/WirelessPhy set L_ 1.0
# 参数设置
set sim(end)            50.0
set val(chan)           Channel/WirelessChannel
set val(prop)           Propagation/TwoRayGround
set val(netif)          Phy/WirelessPhy
set val(mac)            Mac/802_11
set val(ifq)            Queue/DropTail/PriQueue
set val(ll)             LL
set val(ant)            Antenna/OmniAntenna
set val(ifqlen)         100
set val(nn)             3
set val(rp)             DSDV
#仿真器实例与仿真数据存放文件设置
set ns_    [new Simulator]
set tracefd      [open hidden.tr w]
set namfd[open hidden.nam w]
set result       [open hidden.d w]
```

仅用于无线网络技术实验

```
$ns_ trace-all $tracefd
$ns_ namtrace-all-wireless $namfd 200 200
# 设置拓扑并配置无线节点
set topo       [new Topography]
$topo load_flatgrid 200 200
create-god $val(nn)
$ns_ node-config       -adhocRouting $val(rp) \
                       -llType $val(ll) \
                       -macType $val(mac) \
                       -ifqType $val(ifq) \
                       -ifqLen $val(ifqlen) \
                       -antType $val(ant) \
                       -propType $val(prop) \
                       -phyType $val(netif) \
                       -channelType $val(chan) \
                       -topoInstance $topo \
                       -agentTrace ON \
                       -routerTrace ON \
                       -macTrace OFF \
                       -movementTrace OFF
# 实例化节点并配置
for {set i 0} {$i < $val(nn) } {incr i} {
      set node_($i) [$ns_ node]
      $node_($i) random-motion 0
}
$node_(0) set X_ 30.0
$node_(0) set Y_ 150.0
$node_(0) set Z_ 0.0
$node_(1) set X_ 80.0
$node_(1) set Y_ 150.0
$node_(1) set Z_ 0.0
$node_(2) set X_ 130.0
$node_(2) set Y_ 150.0
$node_(2) set Z_ 0.0
$ns_ initial_node_pos $node_(0) 10
$ns_ initial_node_pos $node_(1) 10
$ns_ initial_node_pos $node_(2) 10
# 数据流配置
set udp_1 [new Agent/UDP]
$udp_1 set class_ 1
set sink_1 [new Agent/LossMonitor]
$ns_ attach-agent $node_(0) $udp_1
$ns_ attach-agent $node_(1) $sink_1
$ns_ connect $udp_1 $sink_1
```

仅用于无线网络技术实验

```
set cbr_1 [new Application/Traffic/CBR]
$cbr_1 attach-agent $udp_1
$cbr_1 set type_ CBR
$cbr_1 set packet_size_ 1400
$cbr_1 set rate_ 1Mb
$cbr_1 set random_ false
$ns_ at 1.0 "$cbr_1 start"

set udp_2 [new Agent/UDP]
$udp_2 set class_ 2
set sink_2 [new Agent/LossMonitor]
$ns_ attach-agent $node_(2) $udp_2
$ns_ attach-agent $node_(1) $sink_2
$ns_ connect $udp_2 $sink_2
set cbr_2 [new Application/Traffic/CBR]
$cbr_2 attach-agent $udp_2
$cbr_2 set type_ CBR
$cbr_2 set packet_size_ 1400
$cbr_2 set rate_ 1Mb
$cbr_2 set random_ false
$ns_ at 2.0 "$cbr_2 start"
# 数据记录函数
proc record {} {
        global sink_1 sink_2 result
        set ns [Simulator instance]
        set time 0.5
        set now [$ns now]
        set bytes_1 [$sink_1 set bytes_]
        set bytes_2 [$sink_2 set bytes_]
        puts $result "$now [expr $bytes_1/$time/1000] [expr $bytes_2/$time/1000]"
        $sink_1 set bytes_ 0
        $sink_2 set bytes_ 0
        $ns at [expr $now+$time] "record"
}
# 仿真开始与停止调度
for {set i 0} {$i < $val(nn) } {incr i} {
    $ns_ at $sim(end) "$node_($i) reset"
}
$ns_ at $sim(end) "puts \"NS EXITING...\" "
$ns_ at $sim(end) "stop"
proc stop {} {
    global ns_ tracefd namfd result
    $ns_ flush-trace
    close $namfd
```

仅用于无线网络技术实验

```
        close $tracefd
        exec gnuplot result.gp &
        $ns_ halt
}
puts "Starting Simulation..."
$ns_ at 0.0 "record"
$ns_ run
```

根据上述脚本代码，可按如下操作完成实验：

①使用“**Ctrl+Alt+T**”快捷键打开终端，并用“**cd /home/wiotlab/WIoTLab/exps/lab1**”命令切换路径，接着使用“**touch Hidden_Terminal.tcl**”命令创建脚本文件，将上述代码抄录至该文件中；

②使用命令“**touch result.gp**”创建绘图脚本文件(①中的脚本调用)，并将下述代码抄录到文件中：

```
set term png
set out "result.png"
set key right bottom
set xrange [0:50]
set ylabel "Throughput(KBps)"
set xlabel "Time(s)"
plot "hidden.d" u 1:2 w lp t "Flow 1:n0->n1","hidden.d" u 1:3 w lp t "Flow 1:n2->n1"
```

③执行“**ns Hidden_Terminal.tcl**”命令运行脚本，可得到没有隐藏终端且没有开启RTS/CTS 时两条流的吞吐量性能，如图 1.6 所示。

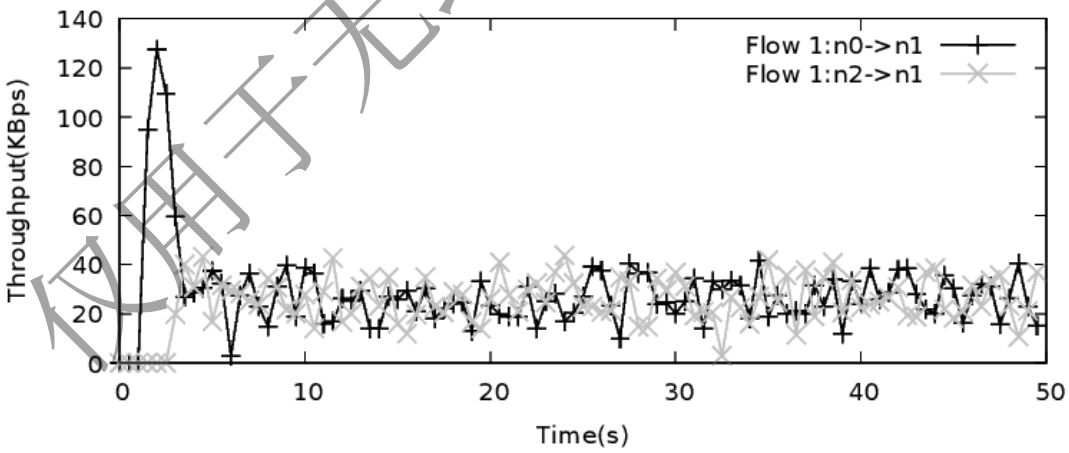


图 1.6　未开启 RTS/CTS 隐藏终端情况下流的吞吐量

④使用命令“**touch pdr.awk**”创建应用层数据包投递率分析脚本，并将下述代码抄录到文件中：

```
BEGIN {
        sendLine = 0;
        recvLine = 0;
```

仅用于无线网络技术实验

```
}
$0 ~/^s.* AGT/ {
    sendLine++;
}
$0 ~/^r.* AGT/{
    recvLine++;
}
END {
        printf "Sent: %d Received: %d, Delivery Ratio:%.4f\n", sendLine, recvLine, (recvLine/sendLine);
}
```

⑤执行命令“**awk -f pdr.awk hidden.tr**”(hidden.tr 为 Hidden_Terminal.tcl 的输出)，可得到结果为：Sent: 17322 Received: 3164, Delivery Ratio:0.1827。可看出，未开启 RTS/CTS 时，应用层数据投递率较低，间接表明冲突较多。

⑥将 Hidden_Terminal.tcl 脚本的“Mac/802_11 set RTSThreshold_ **3000**”修改为“Mac/802_11 set RTSThreshold_ **0**”(此时对所有数据发送开启 RTS/CTS)，重复③和⑤，可得到应用层数据投递情况为：Sent: 17322 Received: 10131, Delivery Ratio:0.5849，且两条流的吞吐量如图 1.7 所示。

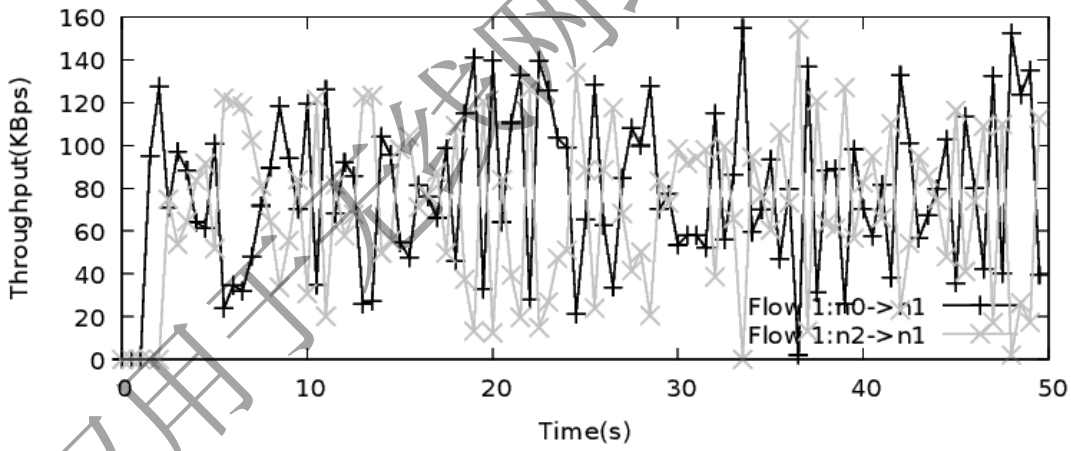
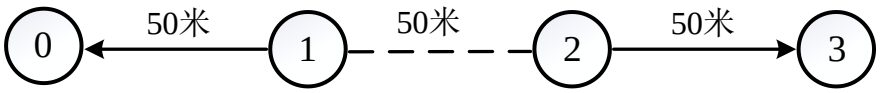


图 1.7　开启 RTS/CTS 隐藏终端情况下流的吞吐量

1.4.2 暴露终端问题

在暴露终端问题仿真实验中，构建了如图 1.8 所示的拓扑，包含 4 个节点且相互间隔 50 米。其中，节点 1 向节点 0 发送数据(CBR/UDP)、节点 2 向节点 3 发送数据(CBR/UDP)。利用前述讨论的 threshold 工具计算得到 50 米的 RXThresh_值为 7.69113e-08，载波侦听 100 米的 CSThresh_值为 1.4268e-08。



仅用于北邮网络技术实验

图 1.8　实验拓扑结构示意图

暴露终端问题仿真分析的实验脚本如下：

```
#无线模块配置
Mac/802_11 set RTSThreshold_ 3000
Mac/802_11 set dataRate_ 2Mb
Antenna/OmniAntenna set X_ 0
Antenna/OmniAntenna set Y_ 0
Antenna/OmniAntenna set Z_ 1.5
Antenna/OmniAntenna set Gt_ 1.0
Antenna/OmniAntenna set Gr_ 1.0
Phy/WirelessPhy set CPThresh_ 10.0
Phy/WirelessPhy set CSThresh_ 1.4268e-08
Phy/WirelessPhy set RXThresh_ 7.69113e-08
Phy/WirelessPhy set bandwidth_ 2e6
Phy/WirelessPhy set Pt_ 0.281838
Phy/WirelessPhy set freq_ 9.14e+6
Phy/WirelessPhy set L_ 1.0
# 参数设置
set sim(end)            50.0
set val(chan)           Channel/WirelessChannel
set val(prop)           Propagation/TwoRayGround
set val(netif)          Phy/WirelessPhy
set val(mac)            Mac/802_11
set val(ifq)            Queue/DropTail/PriQueue
set val(ll)             LL
set val(ant)            Antenna/OmniAntenna
set val(ifqlen)         100
set val(nn)             4
set val(rp)             DSDV
#仿真器实例与仿真数据存放文件设置
set ns_    [new Simulator]
set tracefd     [open hidden.tr w]
set namfd[open hidden.nam w]
set result      [open hidden.d w]
$ns_ trace-all $tracefd
$ns_ namtrace-all-wireless $namfd 200 200
# 设置拓扑并配置无线节点
set topo       [new Topography]
$topo load_flatgrid 200 200
create-god $val(nn)
$ns_ node-config       -adhocRouting $val(rp) \
                       -llType $val(ll) \
                       -macType $val(mac) \
```

仅用于无线网络技术实验

```
                        -ifqType $val(ifq) \
                        -ifqLen $val(ifqlen) \
                        -antType $val(ant) \
                        -propType $val(prop) \
                        -phyType $val(netif) \
                        -channelType $val(chan) \
                        -topoInstance $topo \
                        -agentTrace ON \
                        -routerTrace ON \
                        -macTrace OFF \
                        -movementTrace OFF
# 实例化节点并配置
for {set i 0} {$i < $val(nn) } {incr i} {
        set node_($i) [$ns_ node]
        $node_($i) random-motion 0
}
$node_(0) set X_ 30.0
$node_(0) set Y_ 150.0
$node_(0) set Z_ 0.0
$node_(1) set X_ 80.0
$node_(1) set Y_ 150.0
$node_(1) set Z_ 0.0
$node_(2) set X_ 130.0
$node_(2) set Y_ 150.0
$node_(2) set Z_ 0.0
$node_(3) set X_ 180.0
$node_(3) set Y_ 150.0
$node_(3) set Z_ 0.0
$ns_ initial_node_pos $node_(0) 10
$ns_ initial_node_pos $node_(1) 10
$ns_ initial_node_pos $node_(2) 10
$ns_ initial_node_pos $node_(3) 10
# 数据流配置
set udp_1 [new Agent/UDP]
$udp_1 set class_ 1
set sink_1 [new Agent/LossMonitor]
$ns_ attach-agent $node_(1) $udp_1
$ns_ attach-agent $node_(0) $sink_1
$ns_ connect $udp_1 $sink_1
set cbr_1 [new Application/Traffic/CBR]
$cbr_1 attach-agent $udp_1
$cbr_1 set type_ CBR
$cbr_1 set packet_size_ 1400
$cbr_1 set rate_ 1Mb
```

仅用于无线网络技术实验

```
$cbr_1 set random_ false
$ns_ at 1.0 "$cbr_1 start"

set udp_2 [new Agent/UDP]
$udp_2 set class_ 2
set sink_2 [new Agent/LossMonitor]
$ns_ attach-agent $node_(2) $udp_2
$ns_ attach-agent $node_(3) $sink_2
$ns_ connect $udp_2 $sink_2
set cbr_2 [new Application/Traffic/CBR]
$cbr_2 attach-agent $udp_2
$cbr_2 set type_ CBR
$cbr_2 set packet_size_ 1400
$cbr_2 set rate_ 1Mb
$cbr_2 set random_ false
$ns_ at 2.0 "$cbr_2 start"
# 数据记录函数
proc record {} {
        global sink_1 sink_2 result
        set ns [Simulator instance]
        set time 0.5
        set now [$ns now]
        set bytes_1 [$sink_1 set bytes_]
        set bytes_2 [$sink_2 set bytes_]
        puts $result "$now [expr $bytes_1/$time/1000] [expr $bytes_2/$time/1000]"
        $sink_1 set bytes_ 0
        $sink_2 set bytes_ 0
        $ns at [expr $now+$time] "record"
}
# 仿真开始与停止调度
for {set i 0} {$i < $val(nn) } {incr i} {
    $ns_ at $sim(end) "$node_($i) reset"
}
$ns_ at $sim(end) "puts \"NS EXITING...\" "
$ns_ at $sim(end) "stop"
proc stop {} {
    global ns_ tracefd namfd result
    $ns_ flush-trace
    close $namfd
    close $tracefd
    exec gnuplot result.gp &
    $ns_ halt
}
puts "Starting Simulation..."
```

仅用于无线网络技术实验

```
$ns_ at 0.0 "record"
$ns_ run
```

根据上述脚本代码，可按如下操作完成实验：

①使用“**Ctrl+Alt+T**”快捷键打开终端，并用“**cd /home/wiotlab/WIoTLab/exps/lab1**”命令切换路径，接着使用“**touch Exposed_Terminal.tcl**”命令创建脚本文件，将上述代码抄录至该文件中；

②使用命令“**touch result.gp**”创建绘图脚本文件(①中的脚本调用)，并将下述代码抄录到文件中：

```
set term png
set out "result.png"
set key right bottom
set xrange [0:50]
set ylabel "Throughput(KBps)"
set xlabel "Time(s)"
plot "exposed.d" u 1:2 w lp t "Flow 1:n0->n1","exposed.d" u 1:3 w lp t "Flow 1:n2->n1"
```

③执行“**ns Exposed_Terminal.tcl**”命令运行脚本，可得到没有隐藏终端且没有开启RTS/CTS 时两条流的吞吐量性能，如图 1.9 所示。

④执行命令“**awk -f pdr.awk exposed.tr**”(exposed.tr 为 Exposed_Terminal.tcl 的输出)，可得到结果为：Sent: 17322 Received: 12184, Delivery Ratio:0.7034。

⑤将 Exposed_Terminal.tcl 脚本的“Mac/802_11 set RTSThreshold_ **3000**”修改为“Mac/802_11 set RTSThreshold_ **0**”(此时对所有数据发送开启 RTS/CTS)，重复③和④，可得到应用层数据投递情况为：Sent: 17322 Received: 10291, Delivery Ratio:0.5941，且两条流的吞吐量如图 1.10 所示。

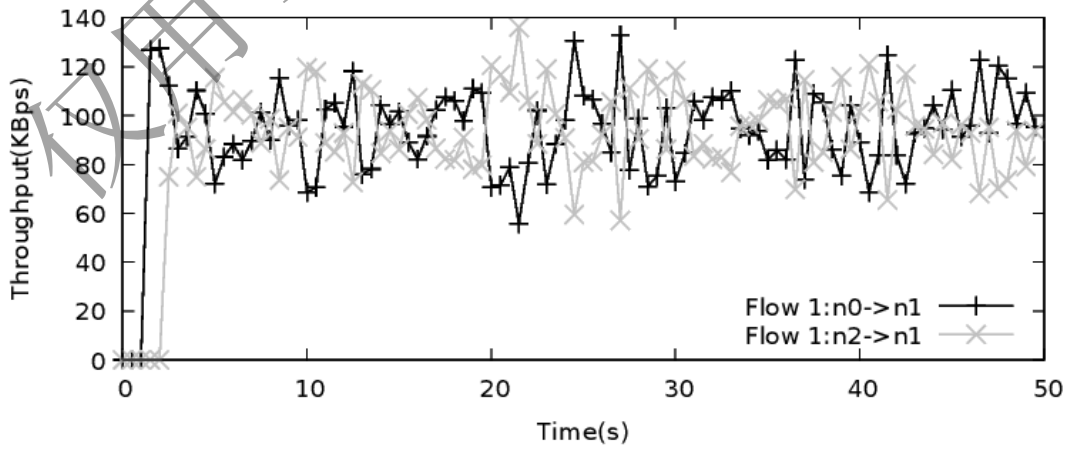


图 1.9　未开启 RTS/CTS 隐藏终端情况下流的吞吐量

仅用于无线网络技术实验

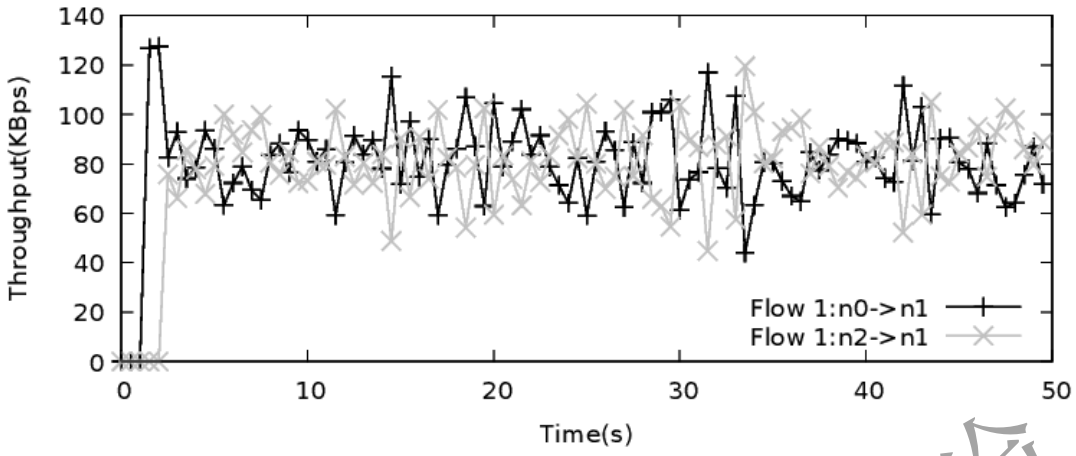


图 1.10　开启 RTS/CTS 隐藏终端情况下流的吞吐量

从上述的实验结果可以看出，暴露终端情况下，开启 RTS/CTS 并未起作用，甚至使得应用层数据包投递率更低。

1.5 结果讨论

由于 IEEE 802.11b/g/n 使用较为广泛，隐藏和暴露终端问题仍然存在。本实验有助于更好的理解 CSMA/CA 的相关技术，如载波侦听、冲突检测。在隐藏终端情况下，RTS/CTS 机制能够起到较好的作用，但在暴露终端时的作用不明显，甚至产生反作用。读者可结合本实验结果和 CSMA/CA 原理进行深入探索。

1.6 扩展练习*

在本实验的基础上，读者可自己动手设计不同网络场景下的仿真，分析网络性能，分析解读实验相关的程序代码。此外，利用实验分析 RTS/CTS 机制对网络性能的提升情况，并给出具体分析结果。

1.7 参考文献

[1]-. IEEE 802.11b-1999. https://en.wikipedia.org/wiki/IEEE_802.11b-1999.

[2] P. C. Ng, S. C. Liew, K. C. Sha, et al. Experimental Study of Hidden-node Problem in IEEE802.11 Wireless Networks. In Proc. SIGCOMM, Aug. 2005, Pennsylvania USA.

[3] L. Wang, K. Wu, M. Hamdi. Combating Hidden and Exposed Terminal Problems in Wireless Networks. IEEE Transactions on Wireless Communications, 2012, 11(11): 4204-4213.

仅用于无线网络技术实验